Especificación, modelado y gestión de requisitos de software

Breve descripción:

Este componente aborda la ingeniería de requisitos y el diseño arquitectónico bajo estándares internacionales. Se profundiza en la especificación mediante IEEE 830, técnicas de priorización, trazabilidad y modelado avanzado con UML y C4. Finalmente, integra herramientas de captura, metodologías del ciclo de vida y control de versiones para asegurar soluciones de software robustas y escalables.

Octubre 2026

Tabla de contenido

Introducción 4

1. Especificación de requisitos de software 6

1.1. Plantillas ERS y estándar IEEE 830 10

1.2. Informe de especificación de requisitos y sus componentes 12

2. Análisis y gestión de requisitos 17

2.1. Análisis de requisitos y técnicas de análisis 21

2.2. Gestión del ciclo de vida de los requisitos 25

3. Priorización y trazabilidad de requisitos 29

3.1. Técnicas de priorización de requisitos 33

3.2. Trazabilidad y matriz de trazabilidad de requisitos 37

4. Modelado en el desarrollo de software 40

4.1. Lenguajes de modelado y técnicas de análisis 46

4.2. UML y modelo C4 para documentación de arquitectura 48

5. Herramientas para la captura y modelado de requisitos 54

5.1. Diagramas de casos de uso, historias de usuario y storyboard 57

5.2. Herramientas de modelado 62

6. Ciclo de vida y control en el desarrollo de software 65

6.1. Control de versiones y herramientas asociadas 70

Síntesis 79

Glosario 81

Referencias bibliográficas 83

Créditos 84

Introducción

En el desarrollo de software, la correcta definición y gestión de los requisitos constituye la base para garantizar que las soluciones construidas respondan de manera efectiva a las necesidades del usuario. La ausencia de una adecuada especificación puede generar inconsistencias, reprocesos y fallos que impactan directamente la calidad del producto final. Por ello, resulta fundamental contar con procesos estructurados que permitan capturar, analizar y gestionar los requisitos de forma clara, precisa y controlada.

Es por ello que este componente aborda estos aspectos desde una perspectiva integral, iniciando con la elaboración de especificaciones formales mediante el uso de plantillas ERS y estándares como el IEEE 830. A partir de esta base, se profundiza en el análisis de requisitos y en su gestión a lo largo del ciclo de vida del software, permitiendo mantener coherencia entre las necesidades del negocio y la solución desarrollada.

Adicionalmente, se incorporan técnicas de priorización y mecanismos de trazabilidad que facilitan el seguimiento de los requisitos, asegurando su cumplimiento durante todo el proceso de desarrollo. El modelado del sistema, mediante lenguajes como UML y el modelo C4, complementa esta visión al proporcionar representaciones visuales que mejoran la comprensión y comunicación entre los actores involucrados.

También se integran herramientas y técnicas para la captura y modelado de requisitos, incluyendo diagramas de casos de uso, historias de usuario y storyboard, fortaleciendo la capacidad de estructurar la información de manera efectiva. Finalmente, se abordan conceptos relacionados con el ciclo de vida del software y el

control de versiones, elementos clave para garantizar la organización, el seguimiento y la evolución controlada de los proyectos.

1. Especificación de requisitos de software

La especificación de requisitos de software es el proceso mediante el cual se documentan de forma estructurada, precisa y verificable las necesidades que debe satisfacer un sistema. Este proceso constituye el punto de partida del desarrollo, ya que define el alcance funcional y no funcional del software, estableciendo una base sólida para su diseño, implementación y validación.

Una especificación adecuada permite traducir las necesidades del usuario en descripciones técnicas comprensibles para el equipo de desarrollo. Esta transformación implica interpretar correctamente los requerimientos del negocio, eliminando ambigüedades y asegurando que cada necesidad quede representada de forma clara dentro del documento.

La especificación no solo describe funcionalidades, sino que también define condiciones bajo las cuales el sistema debe operar. Esto incluye restricciones técnicas, reglas de negocio y atributos de calidad que influyen directamente en el comportamiento del sistema. De esta manera, se establece un marco completo que orienta el desarrollo y reduce la incertidumbre en la toma de decisiones.

Desde una perspectiva estructural, los requisitos se clasifican en dos grandes categorías:

a) **Funcionales**

Describen las acciones, servicios y funcionalidades específicas que el sistema debe realizar para satisfacer las necesidades de los usuarios y los objetivos del negocio, indicando qué hace el sistema en respuesta a determinadas entradas o situaciones.

b) No funcionales

Definen las características de calidad y las restricciones bajo las cuales el sistema debe operar, estableciendo cómo debe comportarse, por ejemplo, en términos de rendimiento, seguridad, usabilidad, disponibilidad y confiabilidad.

Los requisitos funcionales representan las acciones o servicios que el sistema debe ofrecer, mientras que los no funcionales establecen criterios de calidad como rendimiento, seguridad o disponibilidad. Esta diferenciación permite abordar el desarrollo desde una visión integral, asegurando que el sistema cumpla tanto en funcionalidad como en desempeño.

Un aspecto crítico en la especificación es la precisión semántica; es decir, la capacidad de expresar los requisitos de manera inequívoca. Esto implica evitar términos subjetivos como “rápido” o “fácil”, sustituyéndolos por criterios medibles que permitan validar su cumplimiento. Por ejemplo, en lugar de indicar que un sistema debe ser “rápido”, se debe especificar un tiempo máximo de respuesta.

La calidad de la especificación depende de características que garantizan su utilidad dentro del proceso de desarrollo. Estas características no se limitan a la redacción, sino que impactan directamente en la forma en que los requisitos pueden ser implementados y evaluados.

Las características de calidad en la especificación son:

- Facilitar la comprensión.
- Evitar contradicciones.
- Cubrir el alcance total.
- Permitir validación.

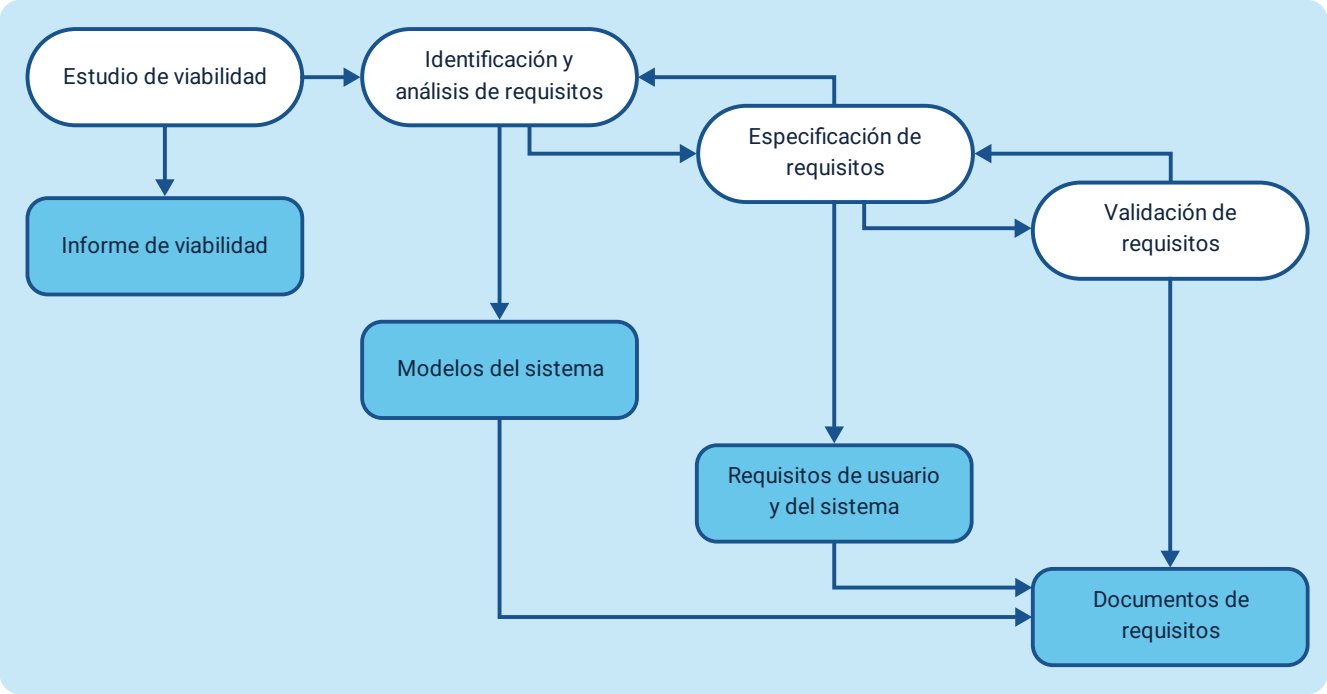
- Relacionar origen y uso.

La trazabilidad, aunque se profundiza en temas posteriores, tiene un rol inicial en la especificación, ya que permite identificar el origen de cada requisito. Esto facilita su control en etapas futuras, especialmente cuando se presentan cambios o ajustes en el sistema.

Otro elemento relevante es la gestión de ambigüedades. Una especificación deficiente puede generar interpretaciones diferentes entre los miembros del equipo, lo que conduce a errores en la implementación. Por ello, se recomienda utilizar estructuras estandarizadas, lenguaje controlado y validaciones constantes con los interesados.

Con base en lo anterior, se presenta el siguiente esquema que integra las principales etapas del proceso de ingeniería de requisitos, desde el estudio de viabilidad hasta la validación de requisitos:

Figura 1. Flujo de transformación de necesidades en requisitos de software



En entornos actuales, la especificación de requisitos se entiende como un proceso evolutivo. A medida que se avanza en el conocimiento del sistema, los requisitos pueden ajustarse, lo que exige una documentación flexible pero controlada. Este enfoque permite adaptarse a cambios sin perder la coherencia del proyecto.

Finalmente, la especificación de requisitos actúa como un punto de referencia transversal dentro del ciclo de vida del software. Su correcta elaboración impacta directamente en la calidad del producto final, ya que influye en el diseño, la implementación, las pruebas y la validación del sistema.

1.1. Plantillas ERS y estándar IEEE 830

La especificación de requisitos de software se apoya en documentos estructurados que permiten organizar la información de forma clara, consistente y verificable. En este contexto, las plantillas ERS y el estándar IEEE 830 cumplen un papel fundamental, ya que proporcionan lineamientos para documentar los requisitos de manera ordenada, facilitando su comprensión, análisis y validación por parte de los diferentes actores del proyecto.

- **Plantillas ERS (Especificación de Requisitos de Software)**

Las plantillas ERS ofrecen una guía práctica para estructurar la documentación de requisitos siguiendo un orden lógico y coherente. Estas plantillas organizan la información en secciones claramente definidas, lo que contribuye a reducir ambigüedades y a mejorar la trazabilidad de los requisitos a lo largo del ciclo de desarrollo.

El documento ERS suele organizarse en secciones que agrupan la información según su propósito, facilitando la lectura y el análisis del sistema a desarrollar. Una estructura típica incluye los siguientes componentes:

a) Introducción

Define el propósito del documento, el alcance del sistema a desarrollar y las principales definiciones, acrónimos y referencias necesarias para una correcta comprensión de los requisitos.

b) Descripción general

Presenta el contexto del sistema, su relación con el entorno operativo y una caracterización general de los usuarios y actores que interactúan con él.

c) Requisitos específicos

Detalla las funcionalidades que el sistema debe proporcionar, así como las restricciones que condicionan su comportamiento y su implementación.

d) Interfaces externas

Describe las interacciones del sistema con otros sistemas, dispositivos o usuarios, especificando los tipos de interfaces y los mecanismos de comunicación involucrados.

Las plantillas ERS no deben entenderse como documentos rígidos o estáticos, sino como herramientas adaptables al contexto del proyecto, al tipo de sistema y al nivel de detalle requerido. En entornos ágiles, por ejemplo, estas plantillas pueden simplificarse para evitar una sobrecarga documental, manteniendo únicamente los elementos esenciales.

- **Estándar IEEE 830**

El estándar IEEE 830 establece criterios reconocidos internacionalmente para la elaboración de documentos de especificación de requisitos de software. Su objetivo principal es asegurar que la documentación sea comprensible, completa y útil para todos los actores involucrados en el proyecto.

Este estándar no impone un formato único, sino que proporciona una base que garantiza la calidad de la especificación. Entre sus principales aportes se encuentra la

definición de las características que debe cumplir una buena especificación de requisitos, tales como:

- Los requisitos deben ser verificables mediante pruebas.
- Deben ser consistentes entre sí.
- Deben estar correctamente estructurados.
- Deben evitar ambigüedades.

Aunque en la actualidad el estándar IEEE 830 ha sido complementado por normas más recientes, como IEEE 29148, su estructura sigue siendo ampliamente utilizada como referencia debido a su claridad y enfoque práctico. Esto permite su aplicación tanto en metodologías tradicionales como en enfoques ágiles, adaptándose a las necesidades específicas de cada proyecto.

En síntesis, el uso de plantillas ERS y el estándar IEEE 830 permite establecer una base sólida para la documentación de requisitos, asegurando claridad, organización y control dentro del proceso de desarrollo. Su correcta aplicación impacta directamente en la calidad del software, al reducir errores de interpretación y facilitar la validación del sistema.

1.2. Informe de especificación de requisitos y sus componentes

El informe de especificación de requisitos de software es el documento formal que consolida y organiza toda la información relevante del sistema desde la perspectiva de lo que debe construirse. A diferencia del punto anterior (estructura estándar), aquí el enfoque está en el documento como entregable real, utilizado para validación, desarrollo y control del proyecto.

Este informe actúa como un contrato técnico entre los stakeholders y el equipo de desarrollo, ya que define de manera precisa el alcance del sistema, sus funcionalidades y las condiciones bajo las cuales debe operar.

Los componentes del informe de especificación de requisitos son similares a los de las plantillas ERS, como se puede apreciar a continuación:

a) Portada

Contiene la identificación del documento, incluyendo el nombre del proyecto, versión, fecha y responsables de su elaboración.

b) Introducción

Expone el propósito del documento, el alcance del sistema y el contexto general en el que se desarrolla el proyecto.

c) Descripción general

Presenta una visión global del sistema, su funcionamiento básico y una caracterización de los usuarios y actores involucrados.

d) Requisitos funcionales

Detalla las funcionalidades y servicios que el sistema debe proporcionar para satisfacer las necesidades de los usuarios.

e) Requisitos no funcionales

Define las restricciones y los atributos de calidad que condicionan el comportamiento del sistema, como rendimiento, seguridad y usabilidad.

f) Interfaces

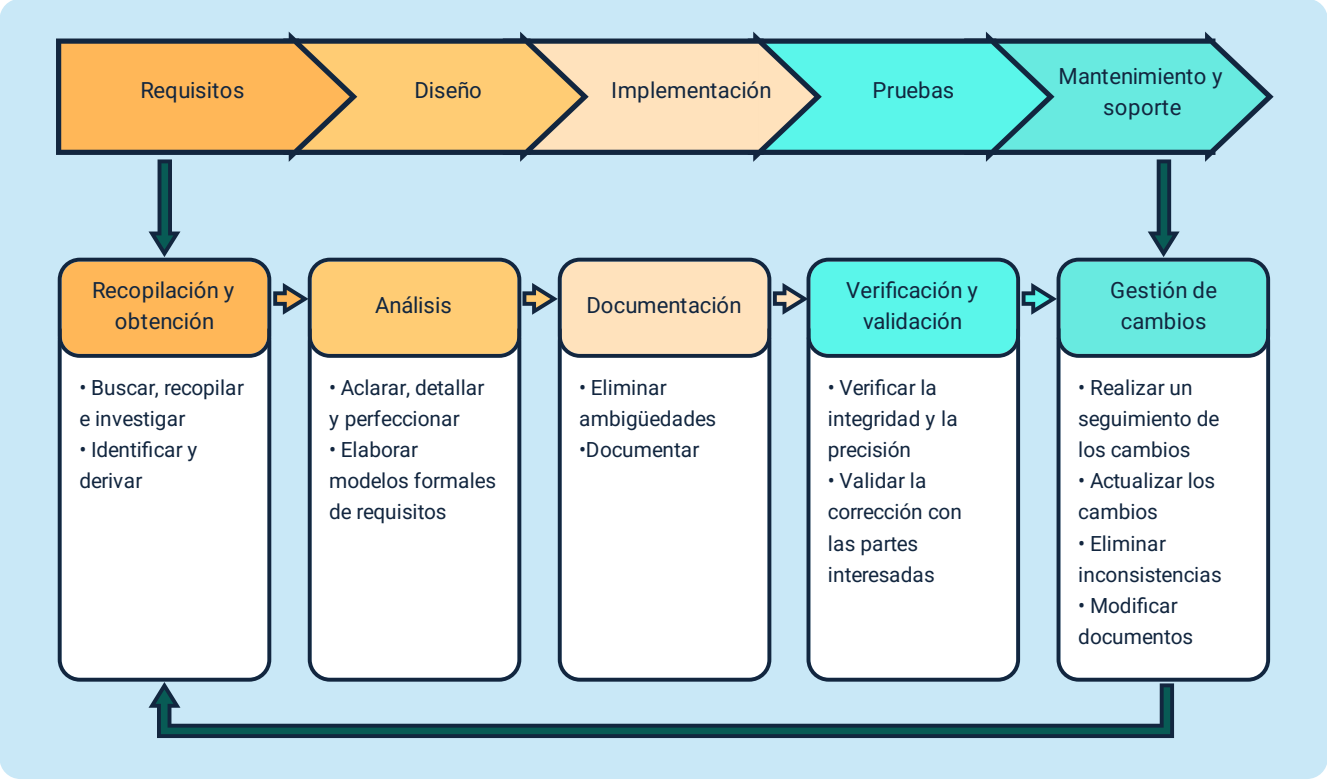
Describe la interacción del sistema con otros sistemas externos, dispositivos o componentes, especificando los mecanismos de comunicación.

g) Anexos

Incluyen información complementaria, como diagramas, glosarios o referencias, que apoyan la comprensión del documento sin sobrecargar el contenido principal.

Con base en lo anterior, la siguiente imagen presenta cómo este documento se integra a las distintas etapas del proyecto de software y apoya las actividades de análisis, validación y gestión a lo largo del ciclo de desarrollo:

Figura 2. Rol del informe ERS dentro del proyecto de software



Cada componente cumple un propósito específico dentro del documento, evitando que la información se mezcle o pierda claridad. Esta organización permite que cada actor consulte únicamente la sección que necesita.

Para finalizar el análisis del informe de especificación de requisitos, se relacionan los aspectos clave que determinan su calidad, su uso a lo largo del proyecto y la manera en que debe mantenerse actualizado, destacando su enfoque práctico dentro del desarrollo de software:

a) Contenido del informe de especificación de requisitos

El valor del informe de especificación de requisitos no radica en su extensión, sino en la precisión y claridad de su contenido. Cada requisito debe estar formulado de tal manera que pueda ser comprendido, implementado y validado sin ambigüedades por todos los involucrados en el proyecto. Un requisito bien definido incluye, como mínimo, un identificador único que permita su trazabilidad, una descripción clara de la necesidad, las condiciones de entrada, el resultado esperado y las restricciones asociadas que delimitan su comportamiento.

b) Uso del informe en el desarrollo del software

El informe de especificación de requisitos no es un documento estático o pasivo, sino una herramienta activa que acompaña el desarrollo del sistema en diferentes etapas. Sirve como base para el diseño del sistema, orienta la construcción de casos de prueba, facilita la validación de los requisitos con el cliente y apoya la gestión de cambios a lo largo del ciclo de vida del proyecto.

c) Control y actualización del informe

Durante el ciclo de vida del software, los requisitos pueden evolucionar, por lo que el informe debe mantenerse actualizado de forma controlada. Esto implica establecer mecanismos de versionamiento del documento, llevar un registro de los cambios realizados y realizar validaciones continuas con los stakeholders, evitando inconsistencias y asegurando la coherencia de la información.

d) Enfoque práctico del informe

En la práctica, el nivel de detalle del informe depende del tipo y contexto del proyecto. Los sistemas críticos requieren una documentación más rigurosa y detallada, mientras que en entornos ágiles se utilizan versiones más ligeras que conservan únicamente los elementos esenciales. En este sentido, el equilibrio entre detalle y utilidad es fundamental, ya que un documento excesivamente extenso puede dificultar su uso, mientras que uno insuficiente puede generar vacíos de información relevantes.

2. Análisis y gestión de requisitos

El análisis y la gestión de requisitos comprenden un conjunto de actividades orientadas a interpretar, organizar, validar y controlar los requisitos del sistema a lo largo del proyecto. Mientras la especificación define qué necesita el sistema, estas fases se enfocan en comprender los requisitos con mayor profundidad, reducir ambigüedades y asegurar que se mantengan alineados con los objetivos del negocio. El análisis permite transformar información inicial, a menudo incompleta o ambigua, en requisitos claros y estructurados, mientras que la gestión controla su evolución mediante el registro, evaluación y aprobación de cambios.

En este contexto, la definición de requisitos se desarrolla como un proceso gradual compuesto por varias actividades interrelacionadas, cuyo propósito es convertir la información obtenida de los usuarios y demás interesados en requisitos comprensibles, consistentes y verificables. A continuación, se describen las principales etapas que conforman este proceso:

a) Elicitación de requisitos

En esta etapa se recopila información relevante mediante técnicas como entrevistas, observación y revisión de documentos, con el fin de comprender las necesidades y expectativas de los usuarios y demás interesados.

b) Análisis de requisitos

La información recopilada se organiza, depura y transforma en requisitos estructurados, identificando relaciones, prioridades y posibles inconsistencias.

c) Validación de requisitos

Se verifica que los requisitos definidos reflejen correctamente las necesidades del usuario y que sean comprensibles para todos los actores del proyecto.

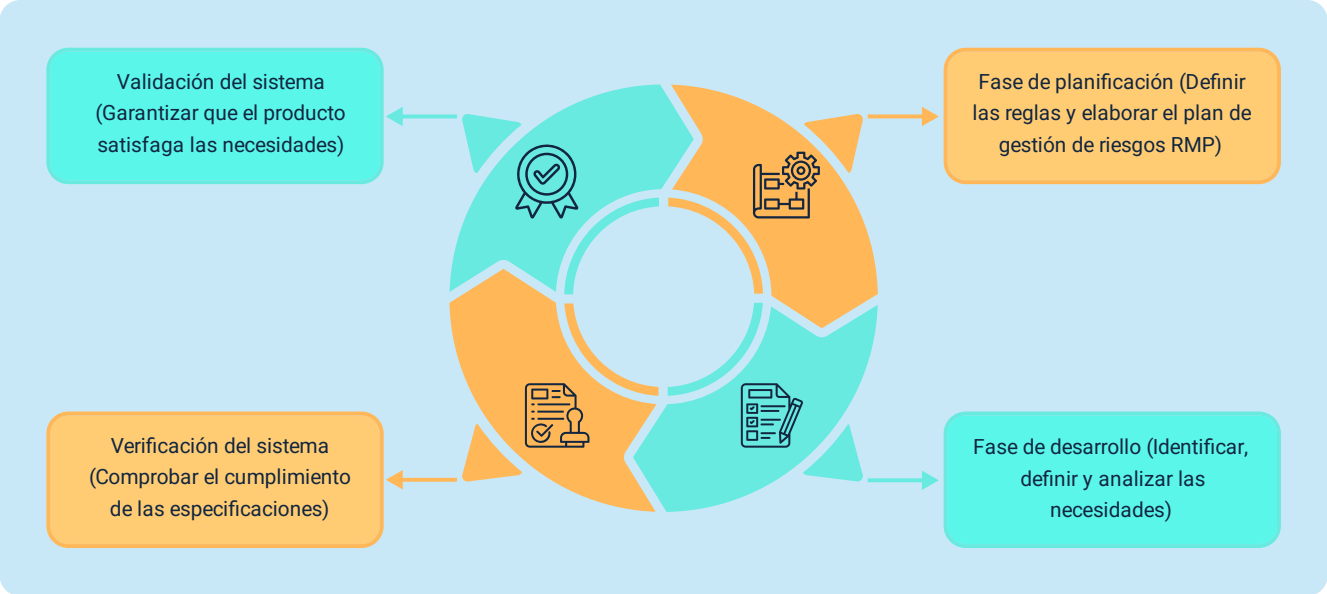
d) Refinamiento de requisitos

Los requisitos se ajustan en su nivel de detalle, precisión y redacción, asegurando que sean claros, completos y verificables.

Este proceso tiene como objetivo principal eliminar ambigüedades y garantizar la calidad de los requisitos. Un análisis adecuado impacta directamente en la calidad del software, ya que errores en estas etapas iniciales pueden propagarse a lo largo de todo el proyecto.

En coherencia con lo expuesto anteriormente, el refinamiento de los requisitos debe entenderse como un proceso progresivo que se desarrolla a lo largo de las distintas fases del proyecto. En este sentido, la siguiente imagen ilustra cómo los requisitos evolucionan desde la planificación y el desarrollo, hasta la verificación y validación del sistema, asegurando su alineación con las necesidades y objetivos definidos:

Figura 3. Refinamiento progresivo de requisitos



Refinamiento progresivo de requisitos

- **Validación del sistema**

(Garantizar que el producto satisfaga las necesidades).

- **Fase de planificación**

(Definir las reglas y elaborar el plan de gestión de riesgos RMP).

- **Fase de desarrollo**

(Identificar, definir y analizar las necesidades).

- **Verificación del sistema**

(Comprobar el cumplimiento de las especificaciones).

Uno de los retos más importantes del análisis es la identificación de conflictos entre requisitos. Estos pueden surgir cuando diferentes stakeholders tienen expectativas distintas o cuando existen limitaciones técnicas. Detectar estos conflictos de manera temprana permite evitar problemas en etapas posteriores.

- **Gestión de requisitos a lo largo del proyecto**

La gestión de requisitos se enfoca en el control y seguimiento de los requisitos durante todo el ciclo de vida del software. A medida que el proyecto avanza, es común que los requisitos cambien debido a nuevas necesidades, ajustes del negocio o restricciones técnicas. La gestión permite manejar estos cambios de forma organizada.

A diferencia del análisis, que se centra en la comprensión inicial, la gestión se orienta a mantener la coherencia del sistema frente a modificaciones. Esto implica registrar cambios, evaluar su impacto y asegurar que todos los involucrados estén alineados con la versión actual de los requisitos.

- **Integración entre análisis y gestión**

El análisis y la gestión de requisitos deben entenderse como procesos complementarios. El análisis asegura que los requisitos estén bien definidos, mientras que la gestión garantiza que se mantengan consistentes a lo largo del tiempo.

Cuando ambos procesos se aplican de manera adecuada, se logra una mayor estabilidad en el desarrollo y se reduce el riesgo de errores. Esta integración permite mantener el control del sistema incluso en entornos donde los cambios son frecuentes.

- **Importancia en el desarrollo de software**

El éxito de un proyecto de software depende en gran medida de la calidad de sus requisitos. Una mala interpretación o un control deficiente puede generar errores que afectan el funcionamiento del sistema y aumentan los costos de desarrollo.

Por el contrario, un adecuado análisis y una gestión eficiente permiten construir sistemas alineados con las necesidades del usuario, mejorar la comunicación entre equipos y facilitar la validación del software. En entornos actuales, estos procesos se integran dentro de metodologías iterativas, donde los requisitos evolucionan de manera controlada. Este enfoque permite adaptarse a cambios sin perder coherencia, garantizando un desarrollo continuo y eficiente.

2.1. Análisis de requisitos y técnicas de análisis

El análisis de requisitos es el proceso mediante el cual se interpretan, organizan y validan las necesidades identificadas durante la elicitación, transformándolas en requisitos claros, coherentes y listos para su especificación. Esta fase es crítica porque actúa como filtro de calidad: detecta ambigüedades, conflictos y vacíos antes de que el sistema sea diseñado o implementado.

En términos prácticos, el análisis no consiste únicamente en “ordenar información”, sino en modelar el problema, comprender el dominio y establecer relaciones entre requisitos. Esto permite construir una visión consistente del sistema, donde cada elemento tiene un propósito definido y puede ser verificado posteriormente.

El siguiente esquema presenta el flujo del análisis de requisitos, desde la información inicial hasta la obtención de requisitos validados:

- Necesidades iniciales.
- Elicitación.
- Análisis.
- Modelado.
- Validación.
- Requisitos estructurados.

a) Necesidades iniciales

Corresponden a las ideas, problemas u objetivos planteados por los usuarios o stakeholders, generalmente expresados de forma informal o poco estructurada.

b) Elicitación

Etapa en la que se recopila información mediante técnicas como entrevistas, observación o revisión de documentos, con el fin de comprender las necesidades reales.

c) Análisis

La información obtenida se examina, depura y organiza para identificar requisitos relevantes, resolver inconsistencias y reducir ambigüedades.

d) Modelado

Los requisitos analizados se representan de forma estructurada mediante modelos, diagramas o descripciones formales que facilitan su comprensión.

e) **Validación**

Se verifica que los requisitos definidos sean correctos, completos y reflejen fielmente las necesidades de los usuarios y del negocio.

f) **Requisitos estructurados**

Resultado final del proceso, donde los requisitos quedan claramente documentados, son comprensibles, verificables y listos para su uso en el desarrollo del sistema.

Durante este proceso, se evalúa si los requisitos son viables desde el punto de vista técnico y si están alineados con los objetivos del negocio. Esto permite evitar desarrollos innecesarios o soluciones que no aportan valor.

El análisis de requisitos tiene como finalidad transformar la información inicial en una base estructurada que permita su implementación. Para ello, se orienta a cumplir los siguientes objetivos:

- **Estructuración de la información**

Organizar los requisitos de manera lógica, agrupándolos según su funcionalidad o relación dentro del sistema.

- **Identificación de dependencias**

Establecer relaciones entre requisitos, determinando cómo uno puede afectar o condicionar a otros.

- **Evaluación de viabilidad**

Analizar si los requisitos pueden ser implementados considerando restricciones técnicas, económicas o de tiempo.

- **Delimitación del alcance**

Definir con precisión qué se incluye dentro del sistema y qué queda fuera, evitando expansión descontrolada del proyecto.

Por su parte, las técnicas de análisis (entrevistas, análisis documental, modelado, escenarios, prototipado) permiten transformar información inicial en requisitos estructurados mediante métodos que facilitan su comprensión y organización. Estas técnicas no sustituyen el criterio del analista, sino que lo apoyan para lograr mayor precisión.

El análisis de requisitos tiene un impacto directo en el éxito del software. Un error en esta fase puede generar problemas que se amplifican en etapas posteriores, aumentando costos y tiempos de desarrollo. Por el contrario, un análisis adecuado permite construir una base sólida para el diseño y la implementación, reduciendo la incertidumbre y mejorando la calidad del sistema.

Este proceso también contribuye a la toma de decisiones, ya que permite identificar qué funcionalidades son realmente necesarias y cuáles pueden descartarse o posponerse.

2.2. Gestión del ciclo de vida de los requisitos

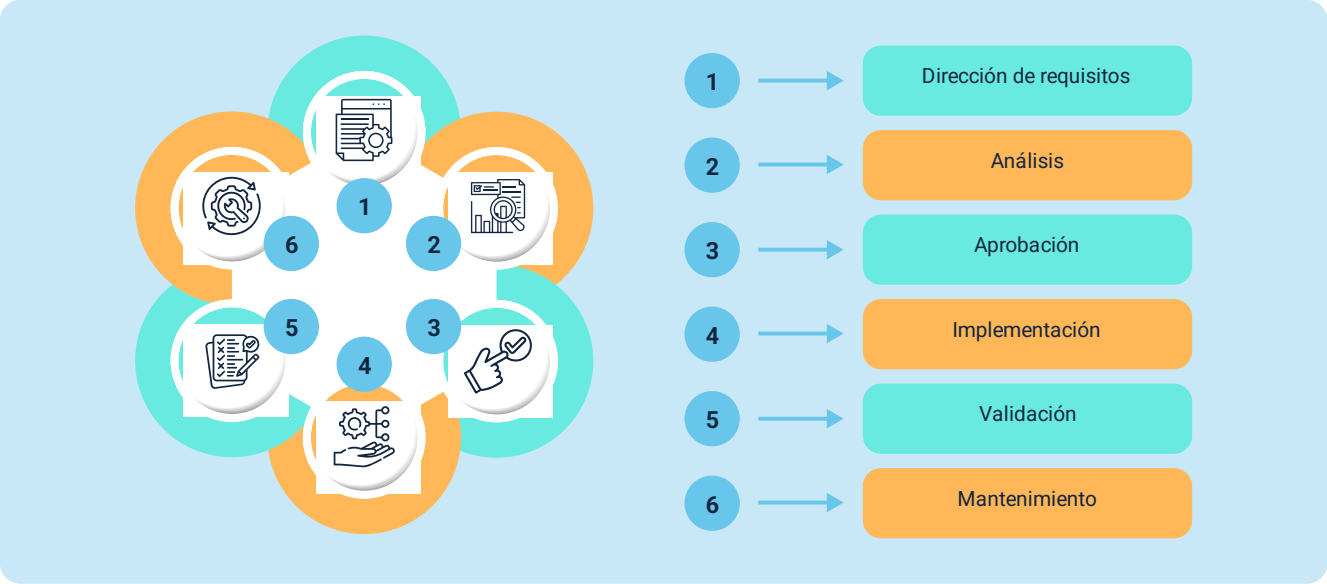
La gestión del ciclo de vida de los requisitos comprende el conjunto de actividades orientadas a controlar, mantener y actualizar los requisitos desde su definición inicial hasta su implementación y validación final. Este proceso permite asegurar que los requisitos se mantengan alineados con los objetivos del proyecto, incluso cuando cambian las condiciones del entorno o las necesidades del negocio.

A diferencia del análisis, que se enfoca en comprender los requisitos, la gestión se orienta a garantizar su estabilidad, trazabilidad y evolución controlada. Esto implica no solo registrar los requisitos, sino también administrar sus cambios, versiones y estado dentro del proyecto.

En la práctica, los requisitos no son estáticos. Durante el desarrollo, pueden surgir nuevas necesidades, ajustes o restricciones, lo que hace necesario contar con un mecanismo que permita integrar estos cambios sin afectar la coherencia del sistema.

La siguiente imagen relaciona el proceso de este ciclo de vida:

Figura 4. Ciclo de vida de los requisitos



Cada requisito atraviesa distintas etapas que permiten controlar su evolución a lo largo del proyecto. Estas etapas no siempre se desarrollan de forma estrictamente lineal, ya que pueden repetirse o ajustarse según la dinámica del desarrollo y las necesidades emergentes. En conjunto, conforman un flujo de control que garantiza que cada requisito sea analizado, aprobado y validado antes de su implementación, reduciendo así el riesgo de incorporar funcionalidades innecesarias, inconsistentes o fuera del alcance definido.

Dentro de este ciclo, la gestión de cambios constituye uno de los elementos más relevantes, dado que los requisitos pueden modificarse por diversas razones, como nuevas necesidades del cliente, cambios en el entorno operativo o limitaciones técnicas identificadas durante el desarrollo. Para evitar desorden y pérdida de control en el

proyecto, cada cambio propuesto debe ser evaluado previamente, considerando su impacto en el sistema, el cronograma y los recursos disponibles.

En este contexto, el control de versiones desempeña un papel fundamental, ya que permite mantener un historial detallado de las modificaciones realizadas sobre los requisitos. Este mecanismo facilita el seguimiento de los cambios y evita la pérdida de información, registrando de manera sistemática qué se modificó, cuándo se realizó el cambio y quién fue el responsable. De esta forma, se asegura la trazabilidad de los requisitos y se apoya el proceso de validación del sistema. Los siguientes son los elementos que hacen parte del control de versiones:

- **Identificador de versión**

Código o número asignado a cada versión del requisito, que permite diferenciar claramente los cambios realizados a lo largo del tiempo y facilita su seguimiento y referencia.

- **Fecha de modificación**

Registro de la fecha en la que se efectuó el cambio, lo cual permite establecer una secuencia temporal de las modificaciones y apoyar la trazabilidad del requisito.

- **Autor del cambio**

Persona o rol responsable de realizar o aprobar la modificación, lo que permite asignar responsabilidades y aclarar el origen de cada ajuste.

- **Descripción del cambio**

Detalle breve y preciso de las modificaciones realizadas, explicando qué se ajustó y el motivo del cambio, con el fin de mantener claridad y control sobre la evolución del requisito.

Este tipo de control resulta especialmente crítico en proyectos complejos, donde múltiples cambios pueden producirse de manera simultánea. Sin una adecuada gestión de versiones, se pierde la trazabilidad de los requisitos, se dificulta su validación y aumenta el riesgo de inconsistencias durante la implementación.

En consecuencia, la gestión del ciclo de vida de los requisitos permite mantener el control del proyecto en escenarios caracterizados por la constante evolución de las necesidades. Una gestión adecuada contribuye a mantener la coherencia del sistema, reducir los riesgos asociados a los cambios, mejorar la planificación del desarrollo y facilitar la validación del software.

Finalmente, en metodologías modernas, este proceso se integra dentro de enfoques iterativos, en los que los requisitos evolucionan de manera continua. Este enfoque posibilita adaptarse a los cambios sin perder control, asegurando que el sistema se mantenga alineado con las necesidades reales de los usuarios y los objetivos del negocio.

3. Priorización y trazabilidad de requisitos

La priorización y trazabilidad de requisitos son procesos complementarios que permiten organizar el valor de los requisitos y asegurar su seguimiento a lo largo del desarrollo del software. Mientras la priorización define qué debe implementarse primero según su importancia, la trazabilidad permite relacionar cada requisito con su origen, implementación y validación.

En proyectos reales, no todos los requisitos pueden desarrollarse al mismo tiempo debido a limitaciones de tiempo, recursos o complejidad técnica. Por esta razón, la priorización se convierte en una actividad clave para tomar decisiones estratégicas, garantizando que las funcionalidades más relevantes se desarrollen primero.

Por otro lado, la trazabilidad asegura que cada requisito pueda ser rastreado durante todo su ciclo de vida, permitiendo verificar su cumplimiento y facilitar la gestión de cambios.

A continuación, se ofrece un poco más detalle de cada una:

a) Priorización

Consiste en clasificar los requisitos según criterios que permiten determinar su importancia dentro del sistema. Este proceso no se basa únicamente en la funcionalidad, sino también en factores como el impacto en el negocio, el riesgo asociado y la dependencia entre requisitos.

Los criterios de priorización de requisitos son:

- **Valor de negocio**: aporte al objetivo del sistema.
- **Urgencia**: necesidad de implementación inmediata.

- **Riesgo**: nivel de impacto si falla.
- **Dependencias**: relación con otros requisitos.
- **Complejidad**: dificultad de desarrollo.

Estos criterios permiten tomar decisiones más informadas, evitando desarrollar funcionalidades de bajo impacto antes que aquellas que aportan mayor valor.

Igualmente, existen diferentes técnicas que permiten aplicar la priorización de manera estructurada. Estas técnicas ayudan a tomar decisiones objetivas, evitando depender únicamente del criterio individual y las más utilizadas o conocidas son:

- **MoSCoW**

Técnica de priorización que clasifica los requisitos según su nivel de importancia para el proyecto, separándolos en imprescindibles, importantes, deseables y prescindibles. Este enfoque ayuda a enfocar los esfuerzos del equipo en aquello que aporta mayor valor y es crítico para el funcionamiento del sistema.

- **Valor vs. esfuerzo**

Método que evalúa cada requisito comparando el beneficio que aporta al negocio con el esfuerzo, costo o complejidad que implica su implementación. Permite identificar rápidamente aquellas funcionalidades que ofrecen alto valor con bajo esfuerzo, optimizando la planificación del desarrollo.

- **Puntuación**

Técnica que asigna valores numéricos a los requisitos con base en criterios previamente definidos, como impacto en el usuario, alineación con los objetivos del

negocio o urgencia. La suma o comparación de estas puntuaciones facilita una priorización más objetiva y transparente.

- **Priorización por riesgo**

Enfoque que prioriza los requisitos considerando los riesgos asociados a su implementación o ausencia, especialmente aquellos que podrían generar mayores impactos negativos en el sistema o el negocio. Este método permite abordar tempranamente los aspectos más críticos y reducir la probabilidad de fallos futuros.

b) Trazabilidad

Permite establecer relaciones entre los requisitos y otros elementos del proyecto, como diseño, desarrollo y pruebas. Este proceso facilita el seguimiento de cada requisito y asegura que todos sean implementados correctamente.

A través de la trazabilidad, es posible identificar el origen de un requisito, su estado actual y su impacto en el sistema. Además, no solo permite verificar el cumplimiento de los requisitos, sino también analizar el impacto de los cambios. Cuando un requisito se modifica, es posible identificar qué otros elementos del sistema se ven afectados.

Al respecto, se tiene que los tipos de trazabilidad de requisitos utilizados son:

- **Trazabilidad hacia adelante**

Permite establecer la relación entre cada requisito y los elementos que lo implementan, como diseño, código, pruebas o componentes del sistema. Facilita verificar que todos los requisitos han sido correctamente desarrollados y validados a lo largo del proyecto.

- **Trazabilidad hacia atrás**

Vincula cada requisito con su origen, ya sea una necesidad del usuario, un objetivo del negocio o una normativa específica. Esta relación ayuda a justificar la existencia de cada requisito y a evaluar el impacto de posibles cambios en las etapas iniciales.

- **Trazabilidad interna**

Describe las relaciones y dependencias entre los propios requisitos, permitiendo identificar cómo un requisito afecta o se relaciona con otros. Esta trazabilidad es clave para mantener la coherencia del sistema y evitar inconsistencias durante el análisis y la gestión de cambios.

La trazabilidad no solo permite verificar el cumplimiento de los requisitos, sino también analizar el impacto de los cambios. Cuando un requisito se modifica, es posible identificar qué otros elementos del sistema se ven afectados.

La priorización y la trazabilidad se complementan dentro del proceso de gestión de requisitos. La priorización define qué se implementa primero, mientras que la trazabilidad permite verificar que cada requisito haya sido correctamente desarrollado.

Cuando ambos procesos se integran, se logra un mayor control del proyecto, ya que se puede planificar el desarrollo y validar su cumplimiento de forma estructurada. Por ello, hay que tener presente:

- La priorización organiza el desarrollo.
- La trazabilidad permite el seguimiento.
- Ambas mejoran la calidad del sistema.

- Reducen riesgos en el proyecto.

La correcta aplicación de estos procesos permite mejorar la eficiencia del desarrollo, garantizar el cumplimiento de los requisitos y facilitar la toma de decisiones. En entornos dinámicos, donde los cambios son frecuentes, la priorización y la trazabilidad se convierten en herramientas clave para mantener el control del proyecto.

3.1. Técnicas de priorización de requisitos

Las técnicas de priorización de requisitos permiten establecer un orden de implementación basado en criterios objetivos, facilitando la toma de decisiones dentro del proyecto. A diferencia de una clasificación general, estas técnicas proporcionan métodos estructurados que permiten comparar requisitos considerando su valor, impacto y viabilidad.

En entornos reales, la priorización no se realiza una única vez, sino que es un proceso continuo. A medida que cambian las condiciones del proyecto, los requisitos deben reevaluarse para garantizar que el desarrollo se mantenga alineado con las necesidades del negocio.

Los factores evaluados en la priorización son:

- **Impacto funcional**: qué tanto aporta al sistema.
- **Frecuencia de uso**: nivel de utilización.
- **Dependencia técnica**: relación con otros requisitos.
- **Esfuerzo de desarrollo**: recursos necesarios.
- **Riesgo asociado**: consecuencias de fallo.

Estos factores permiten identificar qué requisitos deben abordarse primero, evitando decisiones basadas únicamente en percepción o preferencia.

La técnica basada en riesgo prioriza los requisitos considerando la probabilidad de fallo y el impacto que dicho fallo tendría sobre el sistema, el negocio o el usuario final. A diferencia de otras técnicas centradas en valor o esfuerzo, este enfoque se orienta a minimizar consecuencias negativas, especialmente en sistemas críticos.

En este contexto, el riesgo no se define únicamente como la posibilidad de error, sino como una combinación entre: probabilidad de ocurrencia del fallo e impacto del fallo en el sistema o negocio.

En la práctica, este análisis permite identificar qué requisitos deben ser abordados con mayor prioridad, no por su valor funcional, sino por el riesgo que representan si fallan. El nivel de riesgo de un requisito no depende de un solo elemento, sino de múltiples factores que deben evaluarse de manera conjunta como son:

- **Complejidad técnica**

Requisitos con lógica compleja o integración con múltiples sistemas presentan mayor probabilidad de fallo.

- **Frecuencia de uso**

Funcionalidades utilizadas constantemente tienen mayor impacto en caso de error.

- **Dependencias**

Requisitos que afectan a otros incrementan el riesgo sistémico.

- **Criticidad del negocio**

Procesos clave (pagos, autenticación, seguridad) tienen impacto alto.

- **Historial de fallos**

Componentes con errores previos presentan mayor probabilidad de fallo.

La aplicación de esta técnica sigue un flujo estructurado que permite evaluar y priorizar los requisitos de manera objetiva:

a) Identificación de requisitos.

b) Evaluación de probabilidad.

c) Evaluación de impacto.

d) Cálculo del riesgo.

e) Priorización.

a) Identificación de requisitos.

Consiste en listar y describir los requisitos del sistema que serán analizados, asegurando que todos estén claramente definidos antes de iniciar su evaluación.

b) Evaluación de probabilidad

En esta etapa se estima la probabilidad de que cada requisito presente riesgos, considerando factores como complejidad, incertidumbre o dependencia de terceros.

c) Evaluación de impacto

Analiza las consecuencias que tendría un problema asociado a cada requisito, evaluando su efecto sobre el alcance, el costo, el tiempo o la calidad del sistema.

d) Cálculo del riesgo

Se obtiene combinando la probabilidad y el impacto, lo que permite asignar un nivel de riesgo a cada requisito de forma comparable y objetiva.

e) Priorización

Finalmente, los requisitos se ordenan según su nivel de riesgo, enfocando los esfuerzos del proyecto en aquellos que requieren mayor atención y control.

La técnica basada en riesgo tiene una relación directa con el proceso de pruebas, ya que los requisitos con mayor riesgo deben ser probados con mayor intensidad y profundidad, así:

- Requisitos críticos → mayor cobertura de pruebas.
- Requisitos complejos → pruebas detalladas.
- Requisitos sensibles → validación continua.
- Requisitos de bajo riesgo → pruebas básicas.

Esta técnica es especialmente relevante en:

- Sistemas financieros (transacciones, pagos).
- Sistemas de salud (información crítica).
- Sistemas de seguridad (autenticación, acceso).
- Plataformas con alta concurrencia.

A pesar de sus ventajas, esta técnica presenta algunos retos:

- Requiere experiencia para evaluar correctamente el riesgo.
- Puede ser subjetiva si no se utilizan criterios definidos.
- Depende de información previa o conocimiento del sistema.

La priorización basada en riesgo no busca maximizar valor, sino minimizar impacto negativo. Su correcta aplicación permite enfocar el desarrollo en los puntos más críticos del sistema, asegurando mayor estabilidad y reduciendo fallos en producción.

3.2. Trazabilidad y matriz de trazabilidad de requisitos

La trazabilidad de requisitos es la capacidad de rastrear y relacionar cada requisito con los distintos elementos del proyecto, desde su origen hasta su implementación y validación. Este proceso permite mantener visibilidad completa del sistema, asegurando que todos los requisitos definidos sean considerados durante el desarrollo y que cualquier cambio pueda evaluarse de manera controlada.

En proyectos de software, donde los requisitos evolucionan constantemente, la trazabilidad se convierte en un mecanismo clave para garantizar coherencia, control y cumplimiento. Sin ella, resulta difícil determinar si un requisito fue implementado correctamente o si un cambio afecta otras partes del sistema.

El enfoque funcional de la trazabilidad se centra en cómo cada requisito se relaciona con los elementos que permiten su materialización dentro del sistema, asegurando que exista una conexión clara entre lo que se define, lo que se diseña, lo que se desarrolla y lo que se valida.

A diferencia de una trazabilidad meramente documental, este enfoque tiene un carácter operativo, ya que permite responder preguntas clave durante el desarrollo como son:

- ¿Qué parte del sistema implementa este requisito?

- ¿Qué componentes se ven afectados si el requisito cambia?
- ¿Cómo se valida que el requisito fue cumplido?

En este sentido, la trazabilidad funcional actúa como un mecanismo de control que vincula los requisitos con su ejecución real dentro del proyecto.

El enfoque funcional de la trazabilidad puede aplicarse en distintos niveles del sistema, dependiendo del grado de detalle requerido:

- **Nivel alto (funcional)**: relación entre requisitos y funcionalidades generales del sistema.
- **Nivel medio (arquitectónico)**: vínculo con componentes o módulos.
- **Nivel bajo (técnico)**: relación directa con código y pruebas.

Esta segmentación permite adaptar la trazabilidad al tipo de proyecto, evitando sobrecarga de información en sistemas simples o falta de control en sistemas complejos.

La matriz de trazabilidad es una herramienta que permite visualizar de manera estructurada las relaciones entre requisitos y otros elementos del sistema. Se utiliza principalmente para verificar que todos los requisitos han sido considerados durante el desarrollo.

A diferencia de una lista simple, la matriz permite analizar múltiples relaciones de forma simultánea, facilitando la identificación de vacíos o inconsistencias. Además, tiene la siguiente estructura:

Tabla 1. Estructura de una matriz de trazabilidad

ID Requisito	Fuente	Diseño	Desarrollo	Prueba	Estado
R1	Usuario.	✓	✓	✓	Validado.
R2	Negocio.	✓	✓	X	En proceso.
R3	Cliente.	✓	X	X	Pendiente.

Esta estructura permite identificar rápidamente qué requisitos han sido implementados, cuáles están en proceso y cuáles requieren atención.

La matriz de trazabilidad se utiliza como herramienta de control durante todo el proyecto. Su actualización constante permite mantener coherencia entre los requisitos y su implementación. Además, facilita la comunicación entre equipos, ya que permite visualizar el estado del proyecto de forma clara y estructurada.

Cuando un requisito cambia, la trazabilidad permite identificar todos los elementos afectados. Esto evita realizar cambios aislados que puedan generar inconsistencias en el sistema.

La trazabilidad permite mantener control sobre el sistema, asegurando que todos los requisitos sean considerados y validados. Su aplicación mejora la calidad del software y facilita la gestión del proyecto.

En entornos complejos, donde existen múltiples dependencias, la trazabilidad se convierte en una herramienta esencial para evitar errores y garantizar la coherencia del sistema.

4. Modelado en el desarrollo de software

El modelado en el desarrollo de software es el proceso mediante el cual se representa un sistema de forma abstracta, utilizando diagramas y estructuras que permiten comprender su funcionamiento antes de su implementación. A través del modelado, es posible visualizar componentes, relaciones, flujos de información y comportamientos, facilitando la comunicación entre los actores del proyecto.

A diferencia de la documentación textual, el modelado permite expresar conceptos complejos de manera simplificada, reduciendo ambigüedades y mejorando la interpretación del sistema. Este proceso no sustituye el desarrollo, sino que actúa como una herramienta que orienta su construcción.

En entornos modernos, el modelado cumple un papel clave en la definición de arquitecturas, el diseño de sistemas y la validación de requisitos, permitiendo anticipar problemas antes de que se materialicen en el código.

El modelado permite representar el sistema desde diferentes perspectivas, facilitando su comprensión y análisis. Su propósito no es describir cada detalle técnico, sino ofrecer una visión estructurada que apoye la toma de decisiones.

Los propósitos del modelado en software son:

- **Visualización**

Permite comprender el sistema de manera global, facilitando la representación de sus componentes, procesos y relaciones para apoyar el análisis y la toma de decisiones.

- **Comunicación**

Facilita la interacción y el entendimiento entre los diferentes equipos involucrados, permitiendo compartir ideas, aclarar dudas y alinear criterios a lo largo del desarrollo del sistema.

- **Validación**

Permite verificar que los requisitos representen correctamente las necesidades del usuario y los objetivos del proyecto, asegurando que lo definido sea correcto antes de avanzar a etapas posteriores.

- **Diseño**

Contribuye a definir la estructura del sistema, sus componentes principales y las relaciones entre ellos, sirviendo como base para la implementación técnica del software.

- **Documentación**

Se encarga de registrar la arquitectura, los modelos y las decisiones clave del sistema, proporcionando un soporte de referencia para el mantenimiento y la evolución futura del software.

Estos propósitos permiten utilizar el modelado como un puente entre el análisis de requisitos y la implementación del sistema.

El modelado puede aplicarse en diferentes niveles de abstracción, dependiendo del grado de detalle requerido en el proyecto:

- **Conceptual**

Proporciona una visión amplia y abstracta del sistema, enfocándose en los conceptos clave, sus relaciones y el propósito general del software. Facilita el entendimiento inicial del problema y sirve como base para los modelos posteriores.

- **Lógico**

Describe con mayor precisión la estructura y el comportamiento del sistema, especificando componentes, flujos de información y reglas de funcionamiento. Permite analizar cómo interactúan las partes del sistema sin depender aún de una tecnología específica.

- **Físico**

Representa la implementación concreta del sistema, detallando la arquitectura técnica, las plataformas, los recursos de hardware y software, así como la forma en que los componentes lógicos se despliegan en el entorno real.

El modelado se clasifica según qué aspecto del sistema se quiere representar. En la práctica, los modelos se combinan para cubrir visión, estructura, comportamiento, datos e interacción. Cada tipo responde a preguntas distintas del proyecto y utiliza artefactos específicos. Dentro de los tipos de modelado se encuentran:

- **Modelado estructural**

El modelado estructural se enfoca en la organización interna del sistema, definiendo cómo se agrupan los componentes, módulos y clases, así como las relaciones entre ellos. Este tipo de modelado permite establecer una visión clara de la

arquitectura lógica del sistema, facilitando la identificación de dependencias, responsabilidades y límites entre las distintas partes.

Su aplicación es fundamental en la fase de diseño, ya que permite estructurar el sistema antes de su implementación, evitando acoplamientos innecesarios y mejorando la mantenibilidad. A través de este modelado se pueden identificar patrones de diseño, capas arquitectónicas y estructuras modulares que optimizan el desarrollo.

Entre los principales elementos que lo componen se encuentran las clases, interfaces, componentes y paquetes, los cuales se relacionan mediante asociaciones, dependencias o herencias.

- **Modelado de comportamiento**

El modelado de comportamiento describe la dinámica del sistema, representando cómo este responde ante eventos, cómo se ejecutan los procesos y cómo evolucionan sus estados a lo largo del tiempo. Este tipo de modelado permite comprender la lógica interna del sistema y anticipar su funcionamiento en diferentes escenarios.

Se utiliza para representar flujos de trabajo, secuencias de interacción y cambios de estado, lo que facilita la identificación de condiciones, decisiones y posibles errores en la lógica del sistema. Su correcta aplicación permite validar el comportamiento esperado antes de la implementación, reduciendo riesgos en el desarrollo.

Este modelado es especialmente útil en sistemas donde existen múltiples procesos concurrentes o donde el comportamiento depende de condiciones específicas.

- **Modelado de datos**

El modelado de datos se orienta a la definición, organización y relación de la información que el sistema maneja. Este tipo de modelado permite estructurar los datos de manera coherente, estableciendo entidades, atributos y relaciones que reflejan la lógica del negocio.

Su aplicación es clave en el diseño de bases de datos, ya que permite garantizar la integridad, consistencia y eficiencia en el almacenamiento de la información. A través de este modelado se definen reglas como claves primarias, relaciones entre entidades y restricciones que aseguran la calidad de los datos.

Además, facilita la integración entre sistemas, ya que establece una base común para el intercambio de información.

- **Modelado de interacción**

El modelado de interacción representa la forma en que los usuarios, sistemas y componentes se comunican entre sí, permitiendo visualizar cómo se ejecutan las funcionalidades desde la perspectiva del uso.

Este tipo de modelado se enfoca en los flujos de interacción, mostrando la secuencia de acciones que ocurren durante la ejecución de un proceso.

Su principal aporte es facilitar la comprensión del sistema desde el punto de vista del usuario, lo que permite validar requisitos funcionales y asegurar que las funcionalidades definidas respondan a necesidades reales.

Este modelado es fundamental para el diseño de interfaces y la definición de casos de uso, ya que permite representar escenarios completos de interacción.

- **Modelado arquitectónico**

El modelado arquitectónico define la estructura global del sistema, estableciendo cómo se organizan los componentes a gran escala y cómo interactúan entre sí. Este tipo de modelado permite representar la distribución del sistema, incluyendo servicios, capas, módulos y tecnologías involucradas.

Su aplicación es esencial en sistemas complejos, donde es necesario garantizar escalabilidad, rendimiento y mantenibilidad. A través de este modelado se pueden definir arquitecturas como microservicios, sistemas en capas o arquitecturas distribuidas.

Además, permite identificar puntos críticos del sistema, como integraciones externas, flujos de datos y dependencias tecnológicas.

- **Modelado de procesos de negocio**

El modelado de procesos de negocio se centra en la representación de las actividades y flujos operativos de una organización, permitiendo entender cómo funcionan los procesos que el sistema soporta o automatiza.

Este tipo de modelado permite identificar tareas, roles, decisiones y secuencias dentro de un proceso.

Su principal utilidad radica en alinear el desarrollo del software con las necesidades del negocio, asegurando que el sistema responda a procesos reales y no únicamente a funcionalidades técnicas.

Además, facilita la optimización de procesos, permitiendo detectar ineficiencias, redundancias o cuellos de botella antes de la implementación del sistema.

4.1. Lenguajes de modelado y técnicas de análisis

Los lenguajes de modelado y las técnicas de análisis constituyen herramientas fundamentales para representar, interpretar y validar sistemas de software, antes de su implementación. Mientras los lenguajes de modelado proporcionan una forma estandarizada de expresar estructuras y comportamientos, las técnicas de análisis permiten transformar los requisitos en representaciones comprensibles y verificables.

En conjunto, estos elementos permiten construir una visión estructurada del sistema, facilitando la comunicación entre equipos y reduciendo errores durante el desarrollo. Su correcta aplicación no solo mejora la calidad del diseño, sino que también optimiza el proceso de toma de decisiones.

Los lenguajes de modelado son sistemas formales que permiten representar diferentes aspectos de un sistema mediante símbolos, reglas y estructuras definidas. Su principal objetivo es estandarizar la forma en que se describe el sistema, asegurando que todos los involucrados comprendan la información de manera uniforme.

UML es el lenguaje de modelado más utilizado en el desarrollo de software, ya que permite representar tanto la estructura como el comportamiento del sistema. Se compone de diferentes tipos de diagramas que facilitan la visualización de distintos aspectos del sistema:

- **Clases**

Representa la estructura estática del sistema, mostrando las clases, sus atributos, métodos y las relaciones entre ellas, lo que permite comprender la organización interna del software.

- **Casos de uso**

Describe la interacción entre los usuarios y el sistema, identificando las funcionalidades principales y los escenarios en los que el sistema responde a las acciones de los actores.

- **Secuencia**

Muestra el flujo de comunicación entre los distintos componentes del sistema a lo largo del tiempo, detallando el orden en que se intercambian mensajes para ejecutar una funcionalidad.

- **Actividades**

Ilustra los procesos y flujos de trabajo del sistema, incluyendo decisiones, paralelismos y secuencias de acciones, lo que facilita entender la lógica de los procesos.

- **Estados**

Representa los cambios de estado de un elemento del sistema en respuesta a eventos, permitiendo analizar su comportamiento dinámico a lo largo de su ciclo de vida.

UML permite modelar sistemas complejos mediante diagramas que representan relaciones, flujos y comportamientos, facilitando la comprensión del sistema antes de su implementación.

Por su parte, el modelo C4 es una técnica de modelado arquitectónico que permite representar el sistema en diferentes niveles de detalle. Su enfoque progresivo facilita la comprensión tanto para perfiles técnicos como no técnicos.

Las técnicas de análisis permiten transformar los requisitos en modelos estructurados. Estas técnicas ayudan a interpretar la información y representarla de forma clara, permiten construir modelos coherentes que reflejan el comportamiento del sistema y su estructura.

Los lenguajes de modelado y las técnicas de análisis no se utilizan de forma independiente. Las técnicas permiten estructurar la información, mientras que los lenguajes permiten representarla.

Esta integración permite construir modelos que no solo sean visuales, sino también coherentes con los requisitos del sistema y por ello, hay que tener presente:

- Las técnicas organizan la información.
- Los lenguajes la representan.
- Ambos mejoran la calidad del diseño.
- Permiten validar el sistema antes de implementarlo.

Estos elementos se integran dentro de metodologías ágiles, permitiendo adaptar el nivel de detalle del modelado según el contexto del proyecto.

4.2. UML y modelo C4 para documentación de arquitectura

La documentación de arquitectura en software requiere representar el sistema desde diferentes niveles de abstracción, permitiendo comprender tanto su estructura interna como su relación con el entorno. En este contexto, UML (Unified Modeling Language) y el modelo C4 se consolidan como enfoques complementarios que facilitan la representación del sistema de manera clara, estructurada y progresiva.

UML proporciona un conjunto amplio de diagramas que permiten describir tanto la estructura como el comportamiento del sistema, mientras que el modelo C4 se enfoca en representar la arquitectura en niveles jerárquicos, facilitando su comprensión para diferentes tipos de usuarios.

La combinación de ambos enfoques permite construir una documentación completa, donde C4 ofrece una visión general del sistema y UML profundiza en los detalles técnicos.

- **UML en la documentación de arquitectura**

UML es un lenguaje de modelado estándar que permite representar múltiples dimensiones del sistema mediante diagramas especializados. Su principal ventaja radica en la capacidad de describir tanto aspectos estáticos como dinámicos del software.

En el contexto arquitectónico, UML se utiliza para definir la estructura del sistema, identificar componentes y representar interacciones entre ellos. Esto permite construir modelos detallados que sirven como base para la implementación.

- **Modelo C4 en la documentación de arquitectura**

El modelo C4 es un enfoque de representación arquitectónica que organiza la información en cuatro niveles, permitiendo comprender el sistema de forma progresiva. Este modelo facilita la comunicación tanto con perfiles técnicos como no técnicos, ya que presenta la arquitectura desde lo general hasta lo específico.

A diferencia de UML, el modelo C4 no define símbolos estrictos, sino que se basa en una representación clara y simple que prioriza la comprensión del sistema. Los

principios del modelo C4 son la abstracción progresiva, la audiencia dirigida, la consistencia entre niveles y el lenguaje simple.

A continuación, se detalla lo que representa cada nivel de modelo C4 y sus respectivas aplicaciones:

a) Nivel 1. Diagrama de contexto

Representa el sistema como una “caja negra” dentro de su entorno, mostrando usuarios y sistemas externos y cómo interactúan con él.

Uso:

- Alineación con stakeholders no técnicos.
- Definición de límites del sistema.
- Identificación de integraciones externas.

Elementos:

- Sistema principal.
- Actores (usuarios/roles).
- Sistemas externos.
- Relaciones (flujos de información).

Buenas prácticas:

- Nombrar actores por rol (ejemplo: cliente web, sistema de pagos).
- Indicar tipo de interacción (REST, eventos, archivos).
- Mantener 5–9 elementos visibles para claridad.

b) Nivel 2. Diagrama de contenedores

Descompone el sistema en contenedores ejecutables (aplicaciones/servicios), mostrando responsabilidades, tecnologías y comunicaciones.

Uso:

- Definición de arquitectura (monolito, microservicios).
- Decisiones tecnológicas (backend, frontend, DB).
- Diseño de integraciones internas.

Elementos:

- Aplicaciones (web, API, batch).
- Bases de datos / colas / cachés.
- Protocolos de comunicación (HTTP, AMQP, gRPC).

Buenas prácticas:

- Incluir tecnología clave por contenedor (p. ej., API – Node.js).
- Diferenciar almacenamiento (DB vs. caché vs. cola).
- Especificar direcciones de flujo (sincrónico/asíncrono).

c) Nivel 3. Diagrama de componentes

Describe la estructura interna de un contenedor, mostrando cómo se organiza la aplicación en módulos funcionales, sus responsabilidades y las dependencias entre ellos.

Uso:

- Comprender la organización interna de una aplicación.
- Analizar la separación de responsabilidades.
- Detectar dependencias innecesarias o acoplamientos altos.
- Apoyar tareas de mantenimiento y evolución del sistema.

Elementos:

- Unidades funcionales (controladores, servicios, módulos).
- Puntos de acceso a funcionalidades.
- Lógica de negocio.
- Acceso y gestión de datos.
- Relaciones entre componentes.

Buenas prácticas:

- Nombrar componentes según su responsabilidad funcional.
- Mantener bajo acoplamiento y alta cohesión.
- Representar solo componentes relevantes (evitar exceso de detalle).
- Alinear los componentes con patrones arquitectónicos utilizados (MVC, capas, hexagonal).

d) Nivel 4. Diagrama de código

Representa el nivel más detallado del modelo C4, enfocándose en la implementación técnica del sistema. Describe cómo se materializan los componentes mediante clases, métodos y estructuras de código.

Uso:

- Comprender la lógica interna del sistema.
- Facilitar el mantenimiento y la incorporación de nuevos desarrolladores.
- Documentar algoritmos complejos o componentes críticos.
- Apoyar revisiones técnicas y refactorización.

Elementos:

- Estructuras principales del código.
- Funcionalidades específicas.
- Datos asociados a las clases.
- Contratos de implementación.
- Organización del código fuente.

Buenas prácticas:

- Documentar solo partes críticas o complejas.
- Mantener coherencia con los componentes definidos en el nivel 3.
- Evitar duplicar el código; usar el diagrama como apoyo conceptual.
- Actualizar la documentación cuando se realicen cambios significativos.

El modelo C4 facilita la toma de decisiones en aspectos clave como:

- **Distribución del sistema**: monolito vs. microservicios.
- **Integración**: APIs REST vs. eventos.
- **Escalabilidad**: separación de contenedores.
- **Mantenibilidad**: modularidad por componentes.

5. Herramientas para la captura y modelado de requisitos

Las herramientas para la captura y modelado de requisitos apoyan de forma integral el proceso de identificación, organización, representación y validación de la información del sistema. Su uso permite estructurar el trabajo del analista, mejorar la comunicación entre los equipos involucrados y reducir errores derivados de ambigüedades o falta de información.

De manera general, estas herramientas pueden agruparse según la función que cumplen dentro del proceso de ingeniería de requisitos. Cada grupo responde a una necesidad específica, desde la recolección inicial de información hasta su documentación y gestión a lo largo del proyecto.

Los siguientes son los tipos de herramientas utilizados en el desarrollo de software:

a) Herramientas de captura de requisitos

Se utilizan principalmente en las etapas iniciales del proceso y permiten recolectar información directamente de usuarios, stakeholders y fuentes documentales, asegurando que las necesidades del sistema queden registradas de forma clara y completa. Hacen parte las siguientes:

- **Formularios digitales**: facilitan la recolección estructurada de información mediante preguntas definidas previamente.
- **Entrevistas asistidas**: permiten capturar información directa a través de conversaciones guiadas con los interesados.
- **Encuestas**: apoyan el análisis de necesidades cuando se requiere recopilar información de un grupo amplio de usuarios.

- **Notas colaborativas**: permiten registrar y compartir información en tiempo real entre los participantes del proceso.

b) Herramientas de modelado de requisitos

Estas herramientas permiten transformar la información capturada en representaciones visuales del sistema, facilitando su comprensión, análisis y validación por parte de equipos técnicos y no técnicos. Entre ellas se encuentran:

- **Draw.io / Diagrams.net**: utilizada para la elaboración de diagramas generales de procesos, flujos y estructuras.
- **StarUML**: orientada al modelado UML, permite representar casos de uso, clases, secuencias y otros diagramas formales.
- **Lucidchart**: facilita la creación de diagramas de forma colaborativa y en línea.
- **Visual Paradigm**: ofrece funcionalidades avanzadas para el modelado de sistemas y arquitecturas complejas.

c) Herramientas de gestión de requisitos

Apoyan la organización, el seguimiento y el control de los requisitos durante todo el ciclo de vida del proyecto, asegurando trazabilidad, control de cambios y alineación con los objetivos del desarrollo. Se destacan:

- **Jira**: permite gestionar requisitos como elementos de trabajo, integrándolos con tareas y actividades del proyecto.

- **Trello**: ofrece una organización visual de requisitos mediante tableros y tarjetas.
- **Azure DevOps**: integra la gestión de requisitos con el control del desarrollo y las pruebas.
- **Notion**: facilita la documentación estructurada y la organización de la información del proyecto.

d) Herramientas de colaboración

Estas herramientas fortalecen la comunicación y el trabajo conjunto entre los diferentes actores del proyecto, promoviendo la validación continua de los requisitos y la toma de decisiones compartida. Incluyen:

- Plataformas de trabajo colaborativo en línea.
- Sistemas de comunicación y seguimiento de actividades.
- Espacios compartidos para revisión y comentarios sobre los requisitos.

e) Herramientas de documentación

Permiten el registro formal y estructurado de los requisitos, modelos y decisiones tomadas, sirviendo como referencia durante el desarrollo, la validación y el mantenimiento del sistema. Hacen parte:

- Documentos estructurados de especificación de requisitos.
- Repositorios compartidos de información.
- Herramientas de control de versiones para documentos.

5.1. Diagramas de casos de uso, historias de usuario y storyboard

Las técnicas de representación funcional permiten describir cómo interactúan los usuarios con el sistema y cómo se ejecutan las funcionalidades desde una perspectiva práctica. Entre las más utilizadas se encuentran los diagramas de casos de uso, las historias de usuario y los storyboards, los cuales permiten visualizar el sistema desde diferentes niveles de detalle.

Estas herramientas no solo facilitan la comprensión de los requisitos, sino que también permiten validar las funcionalidades con los usuarios antes de su implementación. Su uso conjunto permite construir una visión completa del sistema, combinando estructura, narrativa y experiencia de usuario.

a) Diagramas de casos de uso

Los diagramas de casos de uso permiten representar la interacción entre los actores y el sistema, identificando las funcionalidades principales. Este tipo de modelado se enfoca en describir qué hace el sistema, sin entrar en detalles técnicos.

A través de estos diagramas, se pueden identificar los diferentes roles que interactúan con el sistema y las acciones que pueden realizar. Esto facilita la comprensión del alcance del sistema y permite validar los requisitos funcionales.

Los elementos que hacen parte de los casos de uso son:

- **Actor**

Representa a un usuario, rol o sistema externo que interactúa con el sistema, iniciando o participando en la ejecución de una funcionalidad.

- **Caso de uso**

Describe una funcionalidad específica que el sistema ofrece para satisfacer una necesidad del actor, mostrando el comportamiento del sistema desde la perspectiva del usuario.

- **Relación**

Indica la interacción o vínculo existente entre los distintos elementos del diagrama, como la comunicación entre actores y casos de uso o las dependencias entre funcionalidades.

- **Sistema**

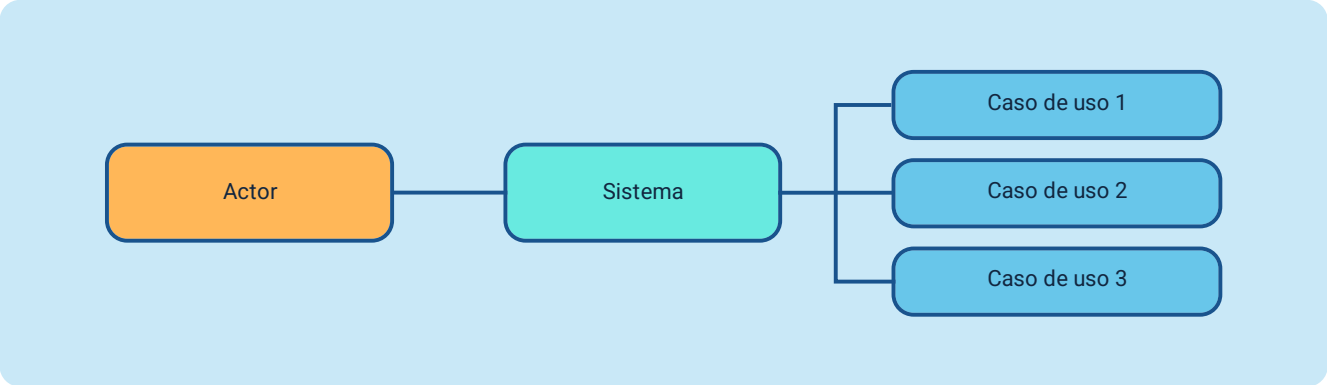
Define el límite del alcance del sistema, delimitando qué funcionalidades forman parte de él y diferenciándolo de los actores y sistemas externos.

- **Extensión**

Representa variaciones o comportamientos alternativos de un caso de uso principal, que se ejecutan bajo determinadas condiciones sin alterar el flujo básico.

A continuación, se ejemplifica la representación de los casos de uso:

Figura 5. Diagrama de casos de uso



b) Historias de usuario

Las historias de usuario son una técnica utilizada principalmente en metodologías ágiles para describir funcionalidades desde la perspectiva del usuario. Estas se redactan en lenguaje natural, facilitando su comprensión y validación.

A diferencia de los casos de uso, las historias de usuario no utilizan diagramas, sino que se enfocan en describir necesidades de forma sencilla y directa.

La estructura de una historia de usuario contiene los siguientes aspectos:

- **Usuario**

Identifica a la persona, rol o perfil que interactúa con el sistema y hace uso de la funcionalidad. Este elemento considera las características, responsabilidades y contexto del usuario, permitiendo que los requisitos se definan de forma alineada con sus necesidades reales y su forma de trabajo.

- **Necesidad**

Describe de manera explícita la acción, objetivo o problema que el usuario desea resolver mediante el sistema. La necesidad expresa qué quiere lograr el usuario y constituye el punto de partida para la definición de la funcionalidad requerida.

- **Beneficio**

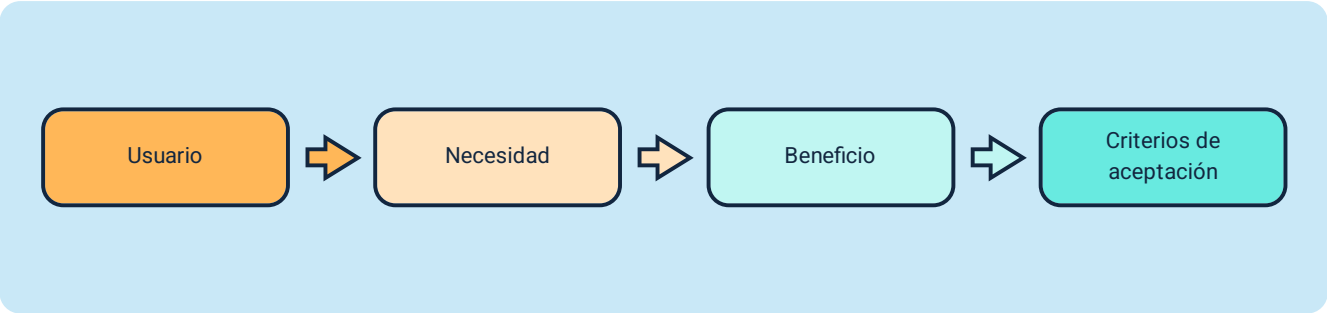
Explica el valor que el usuario obtiene al satisfacer la necesidad, detallando cómo la funcionalidad contribuye a mejorar la eficiencia, reducir errores, optimizar procesos o apoyar la toma de decisiones dentro de su contexto.

- **Criterios**

Corresponden a un conjunto de condiciones claras, específicas y verificables que deben cumplirse para considerar que la funcionalidad ha sido correctamente implementada. Estos criterios permiten validar el comportamiento del sistema y asegurar que cumple con las expectativas y requerimientos del usuario.

Su representación se puede entender de la siguiente manera:

Figura 6. Estructura de una historia de usuario



c) Storyboard

El storyboard es una técnica visual que permite representar la experiencia del usuario a través de secuencias gráficas, mostrando cómo interactúa con el sistema paso a paso. Esta herramienta se utiliza principalmente en el diseño de interfaces y en la validación de flujos de interacción.

A diferencia de los casos de uso y las historias de usuario, el storyboard se enfoca en la experiencia visual, permitiendo identificar problemas de usabilidad antes de la implementación.

Los elementos del storyboard se representan de la siguiente manera:

- **Escena**

Representa una acción o situación concreta dentro de un contexto de uso definido, describiendo el entorno y el momento en que ocurre la interacción.

- **Usuario**

Actor principal que interactúa con el sistema, caracterizado por un rol, necesidades específicas y un objetivo a alcanzar.

- I**nteracción**

Conjunto de acciones realizadas por el usuario sobre la interfaz, como seleccionar, ingresar o confirmar información.

- **Flujo**

Secuencia lógica y ordenada de eventos e interacciones que muestran cómo evoluciona el proceso dentro del sistema.

- **Resultado**

Respuesta generada por el sistema tras la interacción, que puede incluir cambios de estado, retroalimentación visual o confirmaciones para el usuario.

El uso de estas técnicas permite mejorar la calidad de los requisitos, facilitar la comunicación entre equipos y garantizar que el sistema responda a las necesidades del usuario.

En entornos ágiles, estas herramientas se utilizan de forma iterativa, permitiendo validar y ajustar funcionalidades de manera continua.

5.2. Herramientas de modelado

Las herramientas de modelado permiten construir, mantener y comunicar modelos del sistema (UML, BPMN, C4, ER, wireframes) de forma estandarizada y colaborativa. Más allá de “dibujar diagramas”, estas plataformas integran funciones de versionamiento, validación, colaboración y exportación, facilitando que los modelos evolucionen junto con el proyecto.

En la práctica, la elección de la herramienta impacta directamente en la calidad de la documentación, la velocidad de diseño y la consistencia entre equipos. Por ello, no se trata solo de la capacidad de diagramación, sino de su integración con el flujo de trabajo (repositorios, CI/CD, gestión de requisitos) y su soporte a estándares.

A. Capacidades clave de las herramientas de modelado

Las herramientas modernas incorporan un conjunto de capacidades que permiten trabajar los modelos como artefactos vivos del proyecto, encontrando las siguientes:

- **Soporte de notación**

Permite trabajar con distintos lenguajes y estándares de modelado como UML, BPMN, ER, C4 y wireframes, facilitando la representación adecuada según el tipo de sistema o nivel de abstracción requerido.

- **Edición colaborativa**

Habilita el trabajo simultáneo en tiempo real entre varios usuarios, lo que mejora la comunicación del equipo y reduce inconsistencias durante la construcción de modelos.

- **Control de versiones**

Registra el historial de cambios realizados en los modelos, permitiendo comparar versiones, revertir modificaciones y mantener trazabilidad sobre la evolución del diseño.

- **Validación de modelos**

Aplica reglas automáticas para verificar la coherencia, integridad y consistencia de los diagramas, ayudando a detectar errores antes de avanzar a etapas posteriores.

- **Exportación**

Ofrece opciones para generar los modelos en distintos formatos como PDF, PNG, SVG o incluso código, facilitando su documentación, distribución o integración con otros procesos.

- **Integración**

Permite conectar la herramienta con repositorios de código y plataformas DevOps, asegurando alineación entre los modelos, el desarrollo y la gestión del proyecto.

Las herramientas se integran en un flujo que conecta requisitos, diseño y validación, asegurando que los modelos se mantengan actualizados, aplicando las siguientes acciones en los requisitos definidos:

- Selección de herramienta.
- Construcción del modelo.
- Revisión colaborativa.
- Ajustes y versión final.

Los modelos deben tratarse como artefactos gestionados, lo que implica control, actualización y validación continua.

6. Ciclo de vida y control en el desarrollo de software

El ciclo de vida del desarrollo de software representa el conjunto de fases mediante las cuales un sistema es concebido, construido, validado y mantenido. Más allá de una secuencia de etapas, el ciclo de vida constituye un marco que permite organizar el trabajo, establecer controles y garantizar la calidad del producto final.

El control dentro del ciclo de vida implica la aplicación de mecanismos que permiten supervisar el avance, gestionar riesgos, validar resultados y asegurar el cumplimiento de los objetivos del proyecto. Este control no se limita a una fase específica, sino que se integra de manera transversal en todo el proceso de desarrollo.

a) Fases del ciclo de vida del software

El ciclo de vida se estructura en fases que permiten dividir el desarrollo en etapas controladas. Cada una tiene objetivos específicos y genera resultados que sirven de base para la siguiente, los cuales se describen a continuación:

- **Planificación**

Esta fase define la base del proyecto, estableciendo objetivos, alcance, restricciones y recursos. Aquí se determina la viabilidad técnica y económica del sistema. Incluye:

  - Estimación de tiempos y costos.
  - Identificación de riesgos iniciales.
  - Definición del equipo de trabajo.
  - Selección del modelo de desarrollo.

Una planificación deficiente genera impactos en todas las fases posteriores.

- **Análisis**

En esta fase se identifican, documentan y validan los requisitos del sistema. Se busca comprender completamente el problema antes de diseñar la solución. Incluye:

  - Elicitación de requisitos.
  - Análisis de necesidades del usuario.
  - Validación con stakeholders.
  - Definición de criterios de aceptación.

El resultado principal es la base funcional del sistema.

- **Diseño**

El diseño transforma los requisitos en una solución técnica estructurada. Aquí se define la arquitectura, los componentes y la organización del sistema. Incluye:

  - Diseño arquitectónico.
  - Modelado de datos.
  - Definición de interfaces.
  - Selección de tecnologías.

Un diseño sólido reduce errores en desarrollo.

- **Desarrollo**

En esta fase se implementa el sistema, transformando los modelos en código funcional. Incluye:

  - Programación de funcionalidades.
  - Integración de componentes.

- Aplicación de estándares de codificación.
- Documentación técnica.

La calidad del código impacta directamente en la mantenibilidad del sistema.

- **Pruebas**

Se valida que el sistema cumpla con los requisitos definidos, detectando errores antes de su liberación. Incluye:

- Pruebas funcionales.
- Pruebas de integración.
- Pruebas de rendimiento.
- Validación con usuarios.

Esta fase asegura la calidad del producto.

- **Despliegue**

Consiste en la puesta en producción del sistema, asegurando su funcionamiento en el entorno real. Incluye:

- Configuración de entornos.
- Instalación del sistema.
- Migración de datos.
- Capacitación de usuarios.

Un despliegue mal ejecutado puede afectar la adopción del sistema.

- **Mantenimiento**

Es la fase más prolongada del ciclo de vida, donde se corrigen errores y se implementan mejoras. Incluye:

  - Corrección de fallos.
  - Actualización de funcionalidades.
  - Optimización del sistema.
  - Soporte técnico.

El mantenimiento asegura la continuidad operativa.

Cada una de estas fases permite reducir la complejidad del desarrollo, facilitando la organización del trabajo y la identificación de errores en etapas tempranas.

b) Modelos de ciclo de vida

El ciclo de vida puede implementarse mediante diferentes modelos, los cuales determinan la forma en que se ejecutan las fases.

La selección del modelo depende del tipo de proyecto, la complejidad del sistema y la necesidad de adaptación a cambios, teniendo en cuenta lo siguiente:

- Modelos secuenciales → mayor control inicial.
- Modelos iterativos → mayor flexibilidad.
- Modelos ágiles → adaptación continua.
- Modelos híbridos → equilibrio entre control y cambio.

c) Control del proceso de desarrollo

El control del proceso de desarrollo es el conjunto de prácticas, mecanismos y métricas que permiten monitorear, evaluar y ajustar el avance del proyecto de software, asegurando que se cumplan los objetivos definidos en términos de alcance, tiempo, costo y calidad.

Este control no se limita a supervisar tareas, sino que implica establecer un sistema continuo de medición, análisis y toma de decisiones, que permita detectar desviaciones y aplicar acciones correctivas de manera oportuna.

En entornos modernos, el control se implementa como un proceso iterativo que acompaña cada fase del ciclo de vida, integrándose con metodologías ágiles, DevOps y gestión de proyectos.

El control efectivo del desarrollo se basa en varios componentes que trabajan de forma integrada:

- **Planificación base**: define objetivos, tiempos y recursos.
- **Seguimiento**: monitorea el avance real.
- **Medición**: cuantifica el desempeño.
- **Evaluación**: analiza resultados.
- **Corrección**: ajusta desviaciones.

Estos componentes permiten establecer un ciclo de control continuo, donde cada etapa alimenta la siguiente.

El control del desarrollo requiere indicadores que permitan medir el desempeño del proyecto de manera objetiva. Estas métricas permiten identificar problemas antes de que se conviertan en riesgos críticos.

En metodologías ágiles, el control del proceso se realiza de manera continua mediante prácticas específicas de la siguiente manera:

- **Sprint planning**: definición de tareas.
- **Daily meeting**: seguimiento diario.
- **Sprint review**: validación de entregas.
- **Retrospective**: mejora del proceso.

Estos mecanismos permiten mantener un control constante del proyecto sin necesidad de procesos rígidos.

6.1. Control de versiones y herramientas asociadas

El control de versiones es el conjunto de prácticas y herramientas que permiten gestionar los cambios en el código fuente y otros artefactos del proyecto, manteniendo un historial completo de modificaciones, facilitando la colaboración entre desarrolladores y asegurando la integridad del sistema a lo largo del tiempo.

En proyectos de software, múltiples personas trabajan simultáneamente sobre los mismos archivos. Sin un sistema de control de versiones, esto generaría conflictos, pérdida de información y dificultad para rastrear cambios. Por ello, herramientas como Git se han convertido en un estándar, permitiendo trabajar de forma distribuida y controlada.

El control de versiones no solo registra cambios, sino que permite retroceder versiones, comparar modificaciones, gestionar ramas de desarrollo y coordinar equipos, convirtiéndose en un componente esencial del ciclo de vida del software.

A continuación, se describen los principales conceptos del control de versiones, fundamentales para el trabajo colaborativo en proyectos de software:

- **Repositorio**

Es el espacio central donde se almacena el proyecto junto con todo su historial de versiones. Contiene los archivos, las configuraciones y los registros de cambios, y puede ser local o remoto para facilitar el trabajo colaborativo.

- **Commit**

Representa una confirmación de cambios realizada por un desarrollador. Cada commit guarda una "fotografía" del estado del proyecto en un momento específico, incluyendo un mensaje descriptivo que explica qué se modificó y por qué.

- **Historial**

Es el registro cronológico de todos los commits realizados en el repositorio. Permite revisar la evolución del proyecto, identificar cuándo se hicieron cambios específicos y, si es necesario, regresar a versiones anteriores.

- **Branch**

Es una línea de desarrollo independiente que se crea a partir del proyecto principal. Las ramas permiten trabajar en nuevas funcionalidades, correcciones o experimentos sin afectar directamente la versión estable del sistema.

- **Merge**

Es el proceso mediante el cual los cambios realizados en una rama se integran nuevamente en otra, generalmente en la rama principal. El merge permite unificar el trabajo de distintos desarrolladores y consolidar avances en una sola versión del proyecto.

a) Modelo de trabajo con Git

El modelo de trabajo con Git se basa en un sistema distribuido que permite a cada desarrollador trabajar de manera independiente sobre una copia completa del repositorio, facilitando la gestión de cambios y la colaboración en equipo.

A diferencia de otros sistemas centralizados, Git permite realizar modificaciones de forma local y luego sincronizarlas con el repositorio remoto, lo que mejora la eficiencia del desarrollo y reduce la dependencia de una conexión constante.

El flujo de trabajo en Git sigue una secuencia lógica que permite gestionar los cambios de manera controlada:

- **Clonar el repositorio**

Permite obtener una copia completa del proyecto en el entorno local, incluyendo su historial de versiones.

- **Preparar cambios (add)**

Se seleccionan los archivos modificados que serán incluidos en el siguiente registro de cambios.

- **Registrar cambios (commit)**

Se guardan los cambios en el historial local, acompañados de un mensaje que describe la modificación realizada.

- **Sincronizar cambios (push)**

Los cambios locales se envían al repositorio remoto, permitiendo que otros desarrolladores accedan a ellos.

- **Actualizar repositorio (pull)**

Se integran los cambios realizados por otros miembros del equipo, manteniendo el repositorio actualizado.

Las siguientes son acciones destacadas de Git:

- Permite trabajar sin conexión y sincronizar posteriormente.
- Cada cambio queda registrado en el historial.
- Se puede regresar a versiones anteriores en cualquier momento.
- Facilita el trabajo colaborativo sin sobrescribir cambios.

b) Estrategias de ramificación

Las estrategias de ramificación permiten organizar el desarrollo del software mediante la creación de diferentes líneas de trabajo dentro del repositorio. Esto evita que los cambios en desarrollo afecten la versión estable del sistema.

El uso de ramas permite trabajar en funcionalidades, correcciones o versiones específicas de manera independiente, facilitando la gestión del proyecto. Dentro de estas estrategias hacen parte los siguientes procesos:

- **Rama principal (main)**

Contiene la versión estable del sistema. Solo se integran cambios que han sido validados.

- **Rama de desarrollo (develop)**

Se utiliza como base para integrar nuevas funcionalidades antes de pasar a producción.

- **Ramas de funcionalidad (feature)**

Se crean para desarrollar nuevas características sin afectar otras partes del sistema.

- **Ramas de corrección (bugfix)**

Se utilizan para solucionar errores detectados en el sistema.

- **Ramas de versión (release)**

Permiten preparar una versión del sistema antes de su despliegue.

Igualmente, tienen las siguientes acciones destacadas:

- Las ramas permiten trabajar en paralelo.
- Reducen el riesgo de errores en producción.
- Facilitan la organización del desarrollo.
- Permiten integrar cambios de forma controlada.

c) Gestión de conflictos en control de versiones

Los conflictos en control de versiones ocurren cuando dos o más desarrolladores realizan cambios sobre la misma parte del código y el sistema no puede decidir automáticamente cuál versión debe conservarse.

La gestión de conflictos es una actividad clave dentro del control de versiones, ya que permite mantener la coherencia del código y evitar errores en la integración. Se destacan los siguientes tipos:

- **Conflicto de contenido**

Se presenta cuando dos o más desarrolladores realizan cambios distintos sobre la misma línea o bloque de código, generando incompatibilidades que el sistema de control de versiones no puede resolver automáticamente.

- **Conflicto de estructura**

Ocurre cuando se eliminan, renombran o reorganizan archivos o carpetas de forma simultánea, lo que afecta la estructura del proyecto y dificulta la correcta integración de los cambios.

- **Conflicto de integración**

Se produce al combinar ramas de desarrollo con modificaciones divergentes, cuando las diferencias entre ellas impiden una fusión automática y requieren intervención manual para su resolución.

Al igual que las anteriores herramientas, esta también posee unas acciones destacadas:

- Identifica de forma clara las diferencias entre versiones.
- Resuelve manualmente conflictos cuando no es posible la fusión automática.
- Evita la pérdida de cambios realizados por distintos desarrolladores.
- Mantiene la coherencia e integridad del código.
- Facilita la integración correcta de las ramas de desarrollo.

Para finalizar, se presenta un video explicativo que, a través de un caso práctico, ejemplifica el uso del control de versiones y las herramientas asociadas, mostrando cómo se gestionan los cambios, la colaboración en equipo y la integración del trabajo a lo largo del desarrollo de software:

Video 1. Control de versiones y herramientas asociadas



Enlace de reproducción del video

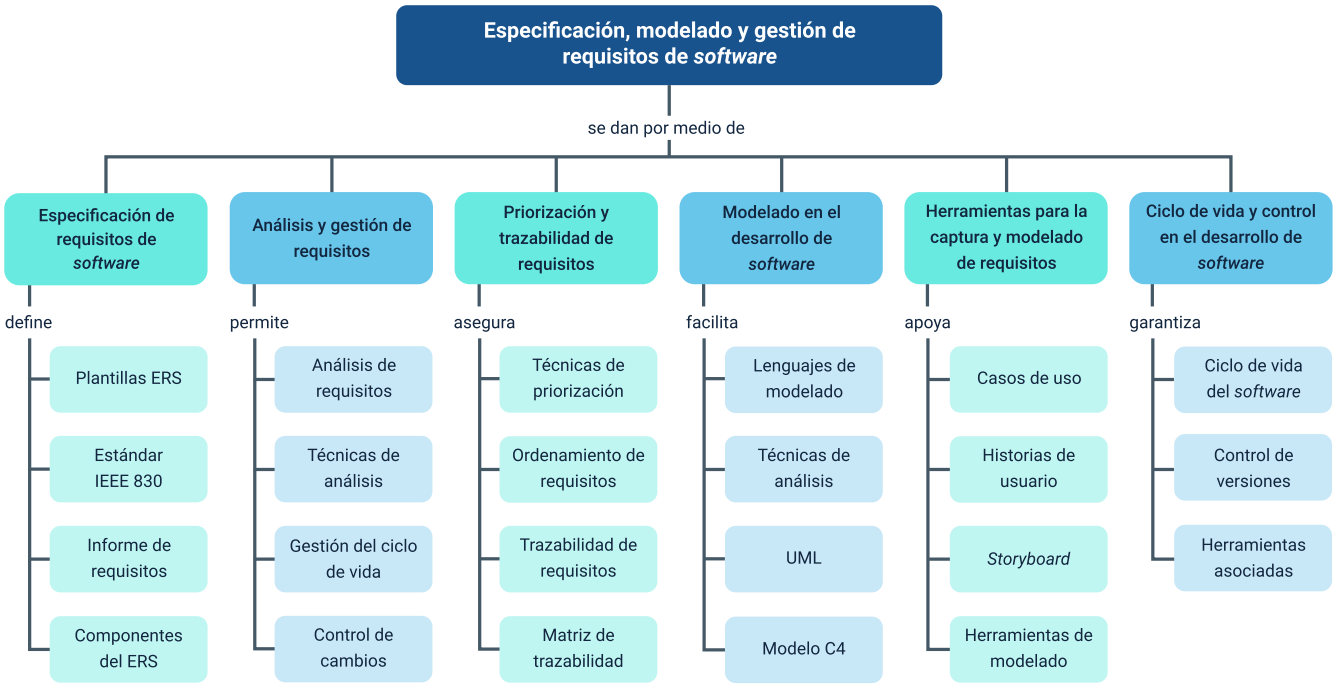
Transcripción del video. Control de versiones y herramientas asociadas
Imagine un caso de desarrollo de software en el que un equipo trabaja de manera colaborativa sobre el mismo proyecto. Al inicio, sin un control de versiones, los integrantes modifican archivos simultáneamente, se sobrescriben cambios y resulta complejo identificar cuál es la versión correcta del sistema. Ante esta situación, el equipo decide incorporar control de versiones como estrategia para gestionar los cambios, mantener un historial confiable y organizar el trabajo conjunto. En este caso, el equipo utiliza Git como herramienta principal. Cada desarrollador obtiene una copia completa del repositorio al clonar el proyecto, realiza ajustes de forma local y registra sus avances mediante un commit, dejando constancia de qué se cambió y por qué. Posteriormente, los cambios se sincronizan

Transcripción del video. Control de versiones y herramientas asociadas
con el repositorio remoto, permitiendo que todos los miembros accedan a la versión más reciente del sistema. Para evitar afectar la versión estable, el equipo trabaja con ramas de desarrollo. La rama principal conserva el sistema funcional, mientras que las ramas de funcionalidad se utilizan para implementar nuevas características. Una vez finalizado y validado el trabajo, los cambios se integran mediante un merge, consolidando los avances en una sola versión. Durante la integración, en este mismo caso, surgen conflictos cuando dos desarrolladores modifican el mismo fragmento de código. La herramienta identifica las diferencias y permite una resolución manual, evitando la pérdida de información y garantizando la coherencia del código. Así, el caso evidencia cómo el control de versiones se convierte en un componente esencial del ciclo de vida del software, al facilitar la colaboración, reducir errores y permitir una evolución controlada y confiable del proyecto.

Síntesis

El componente formativo se centra en la ingeniería de requisitos y el modelado como ejes fundamentales del desarrollo de software, abordando de manera progresiva la especificación, el análisis, la gestión y la priorización de los requisitos. Inicia con el estudio de las plantillas de especificación de requisitos de software (ERS) y los estándares asociados, así como la estructura y componentes del informe de requisitos; posteriormente, profundiza en las técnicas de análisis, la gestión del ciclo de vida de los requisitos y los mecanismos de priorización y trazabilidad que garantizan su coherencia y alineación con los objetivos del proyecto.

De forma complementaria, el componente integra el modelado como herramienta clave para la comprensión y documentación del sistema, mediante el uso de lenguajes y técnicas de modelado, UML y el modelo C4 para la representación de la arquitectura. Finalmente, se abordan las herramientas para la captura y el modelado de requisitos —como casos de uso, historias de usuario y storyboards— junto con los conceptos de ciclo de vida del software y control de versiones, fortaleciendo la gestión de cambios, la colaboración en equipo y la calidad del producto a lo largo del desarrollo.


Especificación, modelado y gestión de requisitos de *software*
se dan por medio de
Especificación de requisitos de *software*
Análisis y gestión de requisitos
Priorización y trazabilidad de requisitos
Modelado en el desarrollo de *software*
Herramientas para la captura y modelado de requisitos
Ciclo de vida y control en el desarrollo de *software*
define
Plantillas ERS
Estándar IEEE 830
Informe de requisitos
Componentes del ERS
permite
Análisis de requisitos
Técnicas de análisis
Gestión del ciclo de vida
Control de cambios
asegura
Técnicas de priorización
Ordenamiento de requisitos
Trazabilidad de requisitos
Matriz de trazabilidad
facilita
Lenguajes de modelado
Técnicas de análisis
UML
Modelo C4
apoya
Casos de uso
Historias de usuario
Storyboard
Herramientas de modelado
garantiza
Ciclo de vida del *software*
Control de versiones
Herramientas asociadas

Glosario

Análisis de requisitos: proceso mediante el cual se examinan, organizan y refinan los requisitos para asegurar su claridad, coherencia y viabilidad antes de su implementación.

Control de versiones: sistema que permite gestionar los cambios en el código fuente, mantener un historial de modificaciones y facilitar el trabajo colaborativo mediante herramientas como Git.

ERS (Especificación de Requisitos de software): documento formal que describe de manera estructurada todos los requisitos del sistema, siguiendo estándares como el IEEE 830, permitiendo su comprensión, validación y desarrollo.

Gestión de requisitos: conjunto de actividades orientadas a controlar, actualizar y mantener los requisitos a lo largo del ciclo de vida del software, garantizando su trazabilidad y consistencia.

Historia de usuario: descripción breve y sencilla de una funcionalidad desde la perspectiva del usuario, utilizada en metodologías ágiles para definir necesidades del sistema.

Modelo C4: enfoque de documentación arquitectónica que organiza la representación del sistema en niveles (contexto, contenedores, componentes y código), facilitando su comprensión progresiva.

Priorización de requisitos: proceso de clasificación de los requisitos según su importancia, impacto y valor para el sistema, con el fin de definir el orden de implementación.

Requisito de software: condición o capacidad que debe cumplir un sistema para satisfacer una necesidad del usuario o del negocio. Constituye la base para el análisis, diseño y desarrollo del software.

Trazabilidad de requisitos: capacidad de relacionar cada requisito con otros elementos del proyecto, como diseño, desarrollo y pruebas, permitiendo su seguimiento y control.

UML (Unified Modeling Language): lenguaje de modelado estandarizado que permite representar la estructura y el comportamiento de un sistema mediante diagramas como clases, casos de uso y secuencia.

Referencias bibliográficas

Booch, G., Rumbaugh, J. & Jacobson, I. (2005). El lenguaje unificado de modelado. Guía del usuario (2.ª ed.). Addison-Wesley.

Brown, S. (2018). Software Architecture for Developers. Leanpub.

Cockburn, A. (2001). Writing Effective Use Cases. Addison-Wesley.

IEEE. (1998). IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications. IEEE.

International Institute of Business Analysis. (2015). BABOK Guide (3rd ed.).

Pressman, R. S. & Maxim, B. (2019). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill.

Sommerville, I. (2016). Ingeniería del software (10.ª ed.). Pearson.

Wiegers, K. & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Javier Mauricio Oviedo	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima